

## 1 Exercise 1 Detection of a coloured ball

### 1.1 a)

We tested the task by using the provided bag files from `kinect_bag_files.zip`. After starting RViz and playing the bag file, we were able to view the video with the colored balls seen in figure 1. The `ball_tracking` framework program was successfully built.

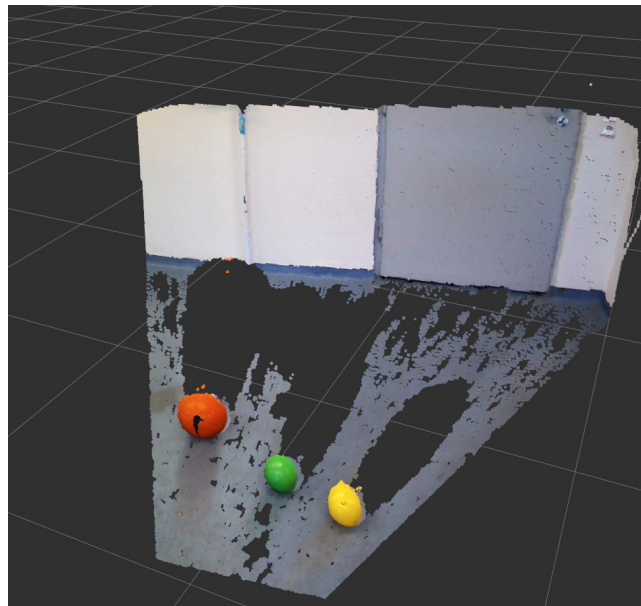


Figure 1: Simulation kinect test

### 1.2 b)

The Microsoft Kinect which is used in this course uses a combination of RGB and depth-sensing technologies to generate colored 3D data.

#### Components

- **IR Projector:** Projects infrared (IR) light patterns onto the scene.
- **IR Camera:** Captures the reflected IR pattern from the scene.
- **Color Camera:** Captures RGB data to provide color information.

## Underlying Principle

- The system employs *matricial active triangulation*, where the IR projector emits coded patterns onto the scene.
- The distortion in these patterns, captured by the IR camera, is analyzed to compute depth information.
- Depth ( $z$ ) is derived using the disparity (horizontal shift) between corresponding points in the projected and captured patterns, as per the disparity-depth mapping equation.

## Combining Data

- The Kinect generates a *disparity map* from the IR images to calculate the 3D geometry.
- RGB data from the color camera is then overlaid onto this geometry, producing colored 3D representations.

## Performance

The Kinect can estimate 3D geometry at 30 frames per second with a spatial resolution of  $640 \times 480$  pixels.

### 1.3 c)

The algorithm is an iterative method designed to estimate the parameters of a mathematical model from a dataset containing a important number of outliers.

First a small, random subset of points from the dataset is selected. This subset must be large enough to define the model, such as two points for a line or three points for a plane. Using this subset, the parameters of the model are estimated.

Next, the algorithm evaluates how well the model fits the entire dataset by calculating the number of points that lie within a certain tolerance of the model. These points are considered consistent with the estimated model.

This process of sampling, estimating, and verifying is repeated for a predetermined number of iterations or until a best model is found. The goal is to identify the model that has the highest number of inliers or best fits the data.

Once the best model is selected, it can be refined by recalculating its parameters using all inliers, for improving its accuracy.

It is very robust against outliers, as it focuses on maximizing agreement among points that support the model while ignoring those that do not.

## 1.4 d)

For this task, two separate functions were implemented: one for detecting and extracting the ground plane as seen in figure 2 and another for walls as seen in figure 3. Both functions utilize the Point Cloud Library (PCL) and the RANSAC segmentation algorithm with plane fitting. The two functions, `findAndExtractFloor` and `findAndExtractWalls`, share a similar structure and follow these steps:

- **Segmentation:** Both use `pcl::SACSegmentationFromNormals` with `SACMODEL_NORMAL_PLANE` to detect planar surfaces.
- **Validation:** A minimum of 500 inliers is required to validate the detected plane.
- **Extraction:** The detected plane points are extracted from the point cloud, and the remaining points are updated for further processing.
- **Publishing:** The extracted planes are published to their respective topics (`/floor` for ground planes, `/walls` for vertical planes).

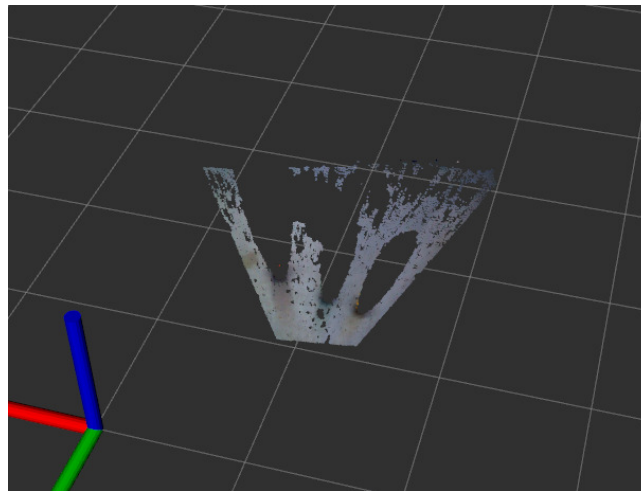


Figure 2: Extract the floor, wall is removed

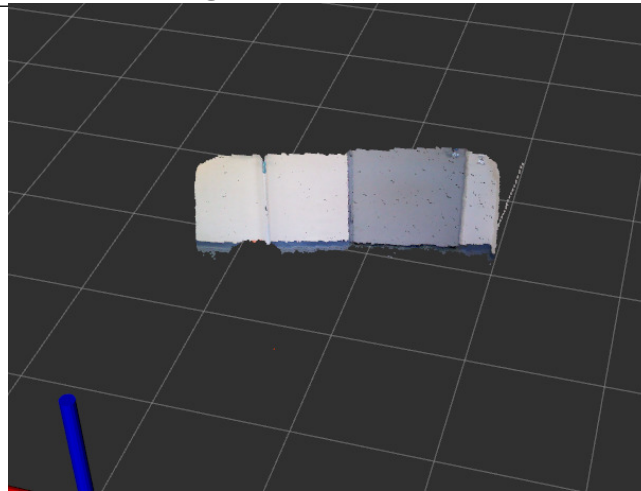


Figure 3: Extract the wall, floor is removed

## 1.5 e)

### Plane Detection

To detect a plane, at least **3 points** are required. A plane in 3D space is described by the equation:

$$ax + by + cz + d = 0$$

This equation has three degrees of freedom ( $a, b, c$ ), as the fourth parameter  $d$  is determined once the normal vector is fixed. Therefore, **3 points** are the theoretical minimum needed. In practice we set this parameter to 1000 points.

In our code, the minimum number of points required for the consensus set is defined indirectly through the configuration of the segmentation algorithms and parameters.

For detecting a floor, the relevant parameters are configured as follows:

- `seg.setDistanceThreshold(0.03)`: This specifies the maximum distance a point can be from the model to be considered an inlier.
- `get_parameter(iterations_plane)`: This sets the number of iterations for RANSAC to find the best model.

For detecting a wall we changed the **DistanceThreshold** to `.3`, as the wall in the ros bag is not flat and needs a higher tolerance to be detected as one plane.

Additionally, the following condition ensures that the detected plane has a robust consensus set:

---

```
if (inliers->indices.size() <= 1000) {  
    PCL_ERROR (Could not estimate a planar model for the given dataset.);  
    return false;  
}
```

Here, at least **1000 points** are required for the consensus set to validate a plane.

## Sphere Detection

To detect a sphere with an unknown radius, at least **4 points** are necessary. A sphere in 3D space is defined by the equation:

$$(x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2 = r^2$$

This equation has 4 parameters: the center of the sphere  $(x_0, y_0, z_0)$  and the radius  $r$ . Thus, **4 points** are required to determine these parameters.

For detecting a sphere, the configuration is defined as:

- `seg.setDistanceThreshold(get_parameter(sphere_distance_threshold))`: Specifies the maximum distance for inliers to the sphere model.
- `get_parameter(iterations_sphere)`: Sets the number of iterations for RANSAC to find the sphere model.

The minimum number of points required for a sphere is controlled by the condition:

```
if (inliers->indices.size() <= 500) {  
    PCL_ERROR (No more spheres found.);  
    continue;  
}
```

This ensures that at least **500 points** must fit the sphere model for a valid ball detection.

## 1.6 f)

The function `findAndExtractBalls` detects and extracts balls from the point cloud as seen in figure 4 using `pcl::SACSegmentationFromNormals` with the model type `pcl::SACMODEL_NORMAL_SPHERE`.

The process iteratively identifies spheres:

- **Segmentation**: For each iteration, the segmentation algorithm identifies points that fit a sphere model, using configurable parameters like the distance threshold and maximum iterations.

- **Validation:** A minimum of 50 inliers is required to validate a sphere.
- **Extraction:** Detected points are extracted, their center is computed, and the sphere is removed from the input cloud for the next iteration.
- **Color:** The function `extractBall` computes the mean color (red, green, blue) of the sphere points and stores the ball's center position in the output cloud and changes the color of the points of the ball color to the average.
- **Publishing:** All detected spheres are published as a point cloud for visualization in RViz.

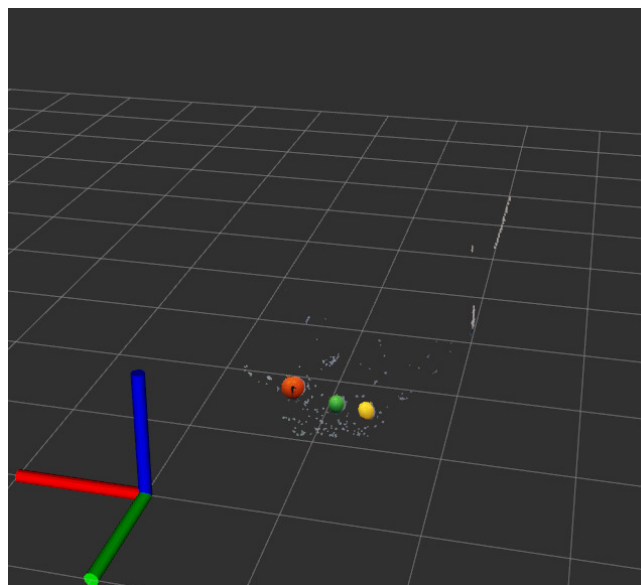


Figure 4: Extract the balls, everything else is removed

In RVIZ, we have multiple PointCloud topics (wall, floor and balls). If we enable all of them, we get most of the original image as shown in figure 5.

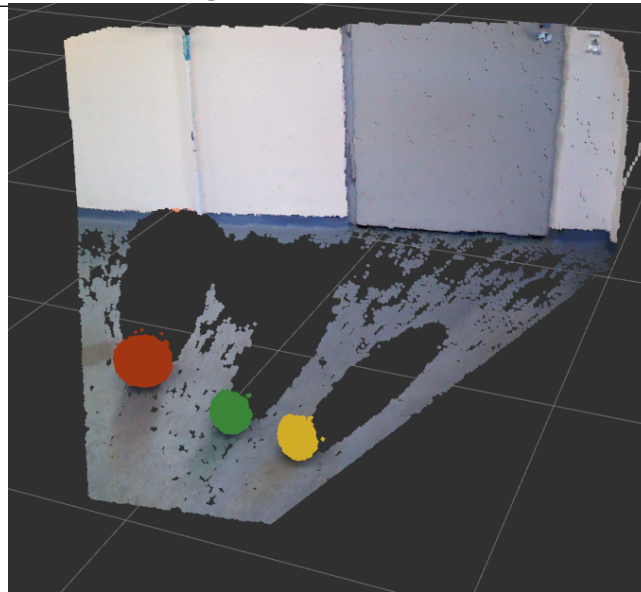


Figure 5: Enable all topics to get the original image

### 1.7 g)

For each detected ball, the algorithm calculates the mean color of the points within its region. Based on this mean value, it determines whether the ball is yellow by evaluating the dominant color component. If a yellow ball is identified, its position is calculated and published as the goal position. The goal is published as a **PoseStamped** message and thus does not point towards anything in particular. This information is then visualized in RViz as seen in figure 7.

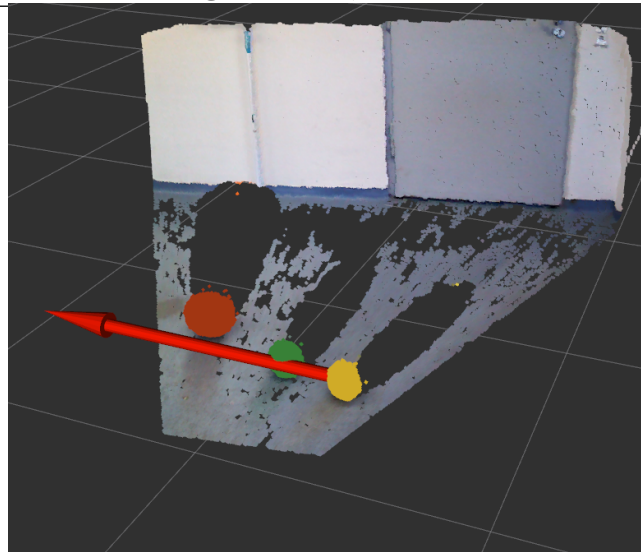


Figure 6: The position of the yellow ball as goal position

### 1.8 h)

When testing the detection with the real robot almost all visible points get detected by the "Wallsplane detector, including the balls and most of the floor. This could possibly be fixed by reducing the distance threshold. The FFloorplane detector only detects some parts of the floor and the "Ballsphere detector does not identify the balls, maybe because their points were already removed from PointCloud.



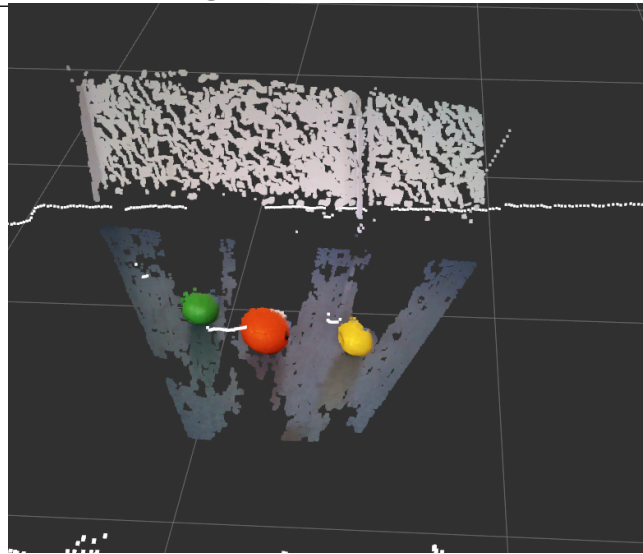


Figure 7